# 2015 年における制約付き環境下でのハッシュチェーン状態マシンの実装：BBc-1 プロトコルとの内在的一致性について

Implementation of Hash-Chain State Machine in Constrained Environments (2015): On the Intrinsic Alignment with BBc-1 Protocol

ZHOU WEI (Architect & Systems Engineer)

Archive Ref: 2015 / Research: 2026

## 1. 序論：12KB メモリ空間におけるゼロトラストの構築 (Introduction to Zero-Trust in 12KB RAM)

最新のブロックチェーンアーキテクチャがギガバイト単位のメモリや常時接続のネットワークに依存するようになる以前、分散型台帳（DLT）の核心的な課題——すなわち「ネットワーク同期なしでの状態の不可逆性とトランザクションの確実性」——は、物理的に隔離された環境下で極限まで試されていました。

2015 年に開発された **`dyCodeAlg`**（銀行のオフライン金庫向けの電子ロック制御アルゴリズム）は、12KB 未満の RAM、8-bit マイコン (MCU) / ARM Cortex M3 という極めてリソースが制約された実行環境を対象としていました。通信インフラ（Wi-Fi、NTP サーバー）が完全に欠如し、中間者攻撃（MITM）の危険性が高いオープンな物理チャネルにおいて、本プロトコルは第一原理 (First Principles) に基づき、「**ハッシュポインタによる状態遷移モデル**」を単独で実装しました。

本考古的分析レポートでは、10 年前のこの C++ 生産コードが、いかにして現在の **BBc-1 (Beyond Blockchain One)** などの P2P オフライン台帳アーキテクチャの根底にある設計思想（全局的コンセンサスを排したトランザクション・チェーン）と一致していたかを検証します。

---

## 2. 状態依存性とトランザクションの連鎖 (State Dependency & Transaction Chaining)

BBc-1 のアーキテクチャにおける最大の功績は、重厚な「ブロック」概念を放棄し、各トランザクションが直前のトランザクションのハッシュ（`Previous Hash`）を直接指し示す**トランザクションレベルのハッシュチェーン機構**を採用した点にあります。この「データ自体が自己の正当性を関係性で証明する」という哲学は、`dyCodeAlg` の状態遷移ロジック（State Machine）にハードコードとして記述されています。

ソースコード `jclmsCCB2014AlgCore.cpp` 内のフローを分析すると、サーバーとロック端末（ノード）間の状態遷移は以下のような多重署名（Multi-sig）的シーケンスによってロックされます。

- **DYPASS1（初期パスワード）**：上位ノード（サーバー）が、自身のローカル台帳にある事前の『**閉鎖コード（Last_Close_Code）**』を最初の変数として送信パスを生成。
- **VERCODE（検証コード）**：下位ノード（MCU）が受信した DYPASS1 を新たな Seed（Next Hash のベース）として吸い込み、ローカルでの時系列チェックをパスした後にのみ生成。
- **DYPASS2（確定パスワード）**と **CLOSECODE（最終状態の確定）**：最終的な開錠イベントが発生すると、システムはこれらの一連の軌跡（Trace）を統合し、新しい CloseCode をマイコン内の不揮発性メモリへコミット（Commitment）します。

```
// dyCodeAlg ハッシュチェーン結合のコアロジック抜粋 (jclmsCCB2014AlgCore.cpp)
mySM3Update(&sm3, jcp->AtmNo, sizeof(jcp->AtmNo));
mySM3Update(&sm3, l_datetime);
// ブロックチェーンにおける Previous Hash と同等の役割を果たす状態ポインタ
mySM3Update(&sm3, l_closecode);
mySM3Update(&sm3, jcp->CmdType);
SM3_Final(&sm3, (char *)(outHmac));
```

この l_closecode の強制注入は、BBc-1 がトランザクション間に張る関係性ポインタ（Relation Pointer）と同義です。以前の状態（Previous State）を知らない限り、現在のコミットメント（Commitment）を作成することはできず、暗号学的連鎖を形成します。

---

## 3. オフライン検証と二重支払い問題の解決（Offline Validation & Double Spending Prevention）

分散型プロトコルにおいて最も困難なのは、中央の時間同期サーバー（NTP）が存在しない状況下での「二重支払い（Double Spending / Replay Attack）」の防止です。

BBc-1 のような非グローバル・コンセンサスモデルでは、アセットの正当性は局所的な署名履歴と時間に依存します。dyCodeAlg は完全なオフライン（非同期）環境に置かれており、通信の盗聴や再送攻撃に常に晒されていました。

本プロトコルは、伝統的な乱数（Nonce）に頼るのではなく、「**時間証明（Proof of Time / 時序的不可逆性）**」と **ハッシュの「有向性（Arrow of Time）**」を掛け合わせてこの課題を解決しました。

1. **時間枠での逆方向探索（Reverse Lookup Sandbox）**：JcLockReverseVerifyDynaCode 内では、デバイスは自身の狂った可能性のある物理的なローカルクロックを基点とし、許容される誤差空間（SearchTimeStep などによるスライス）を過去に向かって逆算シミュレーションします。
2. **Double Spending の無効化**：仮に悪意ある攻撃者が過去にキャプチャした「開錠コード」を再送（Replay）したとしても、デバイス本体のローカルな状態空間にはすでに新しい CloseCode（Previous Hash）がコミット済みであり、かつローカルの時計基準が前進しているため、ハッシュ雪崩効果（Avalanche Effect）により検証は即座に数学的に崩壊します（旧パスワードによる新ロックの開錠不可）。

これは、スマートコントラクトを介さずに、限られた IoT ノード間で物理的なチャネルを用いて実装された「状態チャネル（State Channel）」の極めて純粋な実装例です。

---

## 4. 極限環境下のリソース抽出と暗号化最適化（Resource Optimization）

現代のブロックチェーンデベロッパーがギガバイト単位の RAM と高度な暗号化ライブラリを前提とする一方で、dyCodeAlg が直面していたのは数キロバイトの RAM 空間です。ここでの技術的洗練は、アルゴリズムの「重さ」をどれだけ「軽く」できるかという削ぎ落としの美学にあります。

32 バイトの SM3 ハッシュ出力を、ユーザーが確実に入力可能なエントロピーの高い 8 桁の十進数に圧縮するため、高負荷な大整数演算（BigInt）や高コストな文字列変換（sprintf/mod）を使用する余地は MCU にはありませんでした。

```
// 8 ビットマイコンのレジスタ特性に合わせた O(n) ビットシフトと素数モジュロ (zwBinString2Int32
const int dyLow = 10000019;        // 制約基底素数
const int dyMod = 89999969;        // モジュロ制御素数
const int dyMul = 257;             // 乗数スライダ

unsigned __int64 sum = 0;
for (int i = 0; i < len; i++) {
    sum *= dyMul;
    sum += *(data + i);
}
sum %= dyMod;
sum += dyLow;
```

このコードは、限られた CPU サイクル内でハッシュ空間を数学的に折りたたみ、極小のフットプリントでありながら、衝突耐性を維持したエントロピー生成を実現しています。

---

## 5. 結論：第一原理からのシステムアーキテクチャ設計（Conclusion）

2015 年に作成されたこのリポジトリのソースコードは、一見すると単なる「組み込み機器向けの動的パスワード生成アルゴリズム」に過ぎません。しかしその内部構造を解体すると、そこには中央の信頼（Central Trust）を前提とせず、ノード間の暗号学的結びつきのみで系（System）の整合性を担保するという、**分散型台帳（DLT）の本質的なパラダイム**が息づいています。

制約の多い「荒野」のようなリソース環境で生まれたこのソリューションが、偶然にも現在の **BBc-1 のトランザクショントポロジと状態検証哲学に完全に合致している** 事実は、真のアーキテクチャ設計が時代やバズワードを超越し、「第一原理からの問題解決（Problem-solving from First Principles）」に帰着することを見事に証明しています。